

Bases de dades no relacionals

TASK FOR BD07

DEYANIRA TORRELLAS

ENLACE

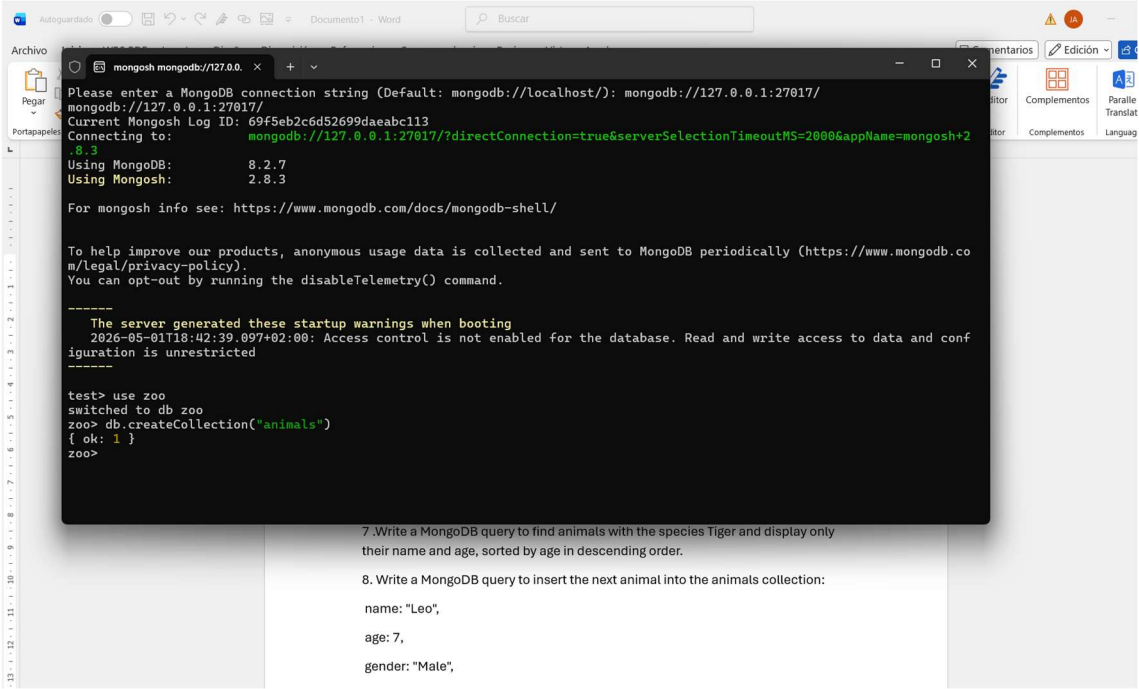
<https://github.com/Deya2609/Base-de-datos/blob/main/Task7.%20Mongo%20db>

VIDEO LOOM

<https://www.loom.com/share/f49d17b67dfa4ab7bdcb7a28c8f0e325>

Write the code used to perform the following tasks from the MongoDB shell:

1. Create a database called zoo.



```
mongosh mongodb://127.0.0.1:27017/
Please enter a MongoDB connection string (Default: mongodb://localhost/): mongodb://127.0.0.1:27017/
mongodb://127.0.0.1:27017/
Current Mongosh Log ID: 69f5eb2c6d52699daeabc113
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 8.2.7
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-05-01T18:42:39.097+02:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

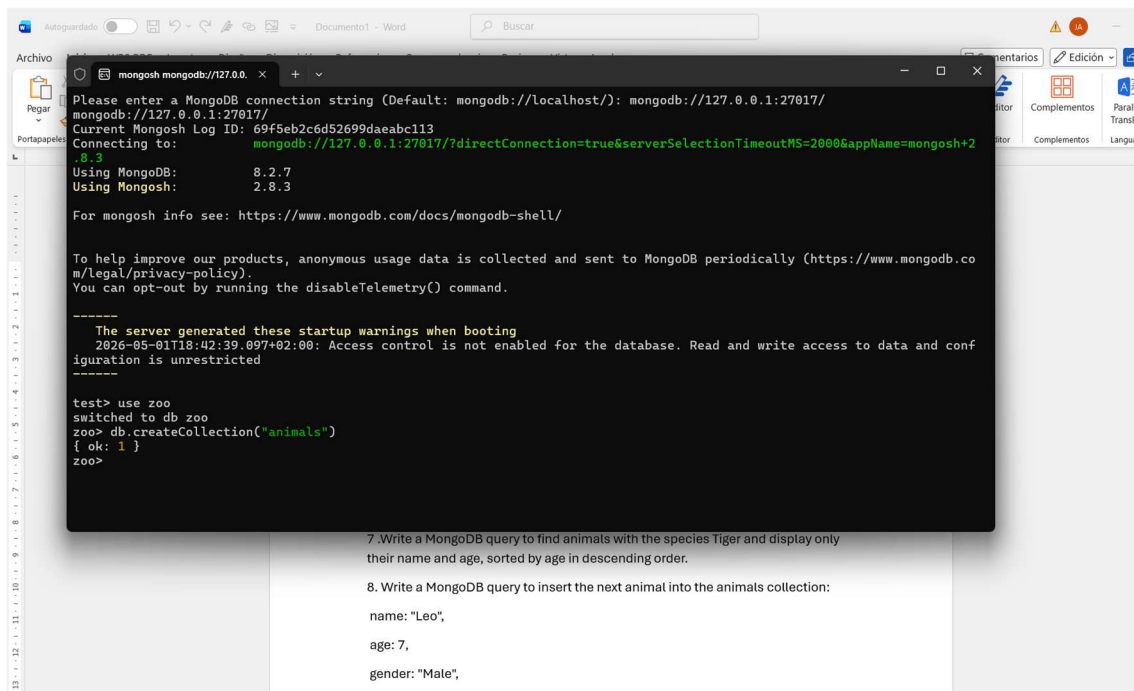
test> use zoo
switched to db zoo
zoo> db.createCollection("animals")
{ ok: 1 }
zoo>
```

7. Write a MongoDB query to find animals with the species Tiger and display only their name and age, sorted by age in descending order.

8. Write a MongoDB query to insert the next animal into the animals collection:

```
name: "Leo",
age: 7,
gender: "Male",
```

2. Insert a collection called animals.



7. Write a MongoDB query to find animals with the species Tiger and display only their name and age, sorted by age in descending order.

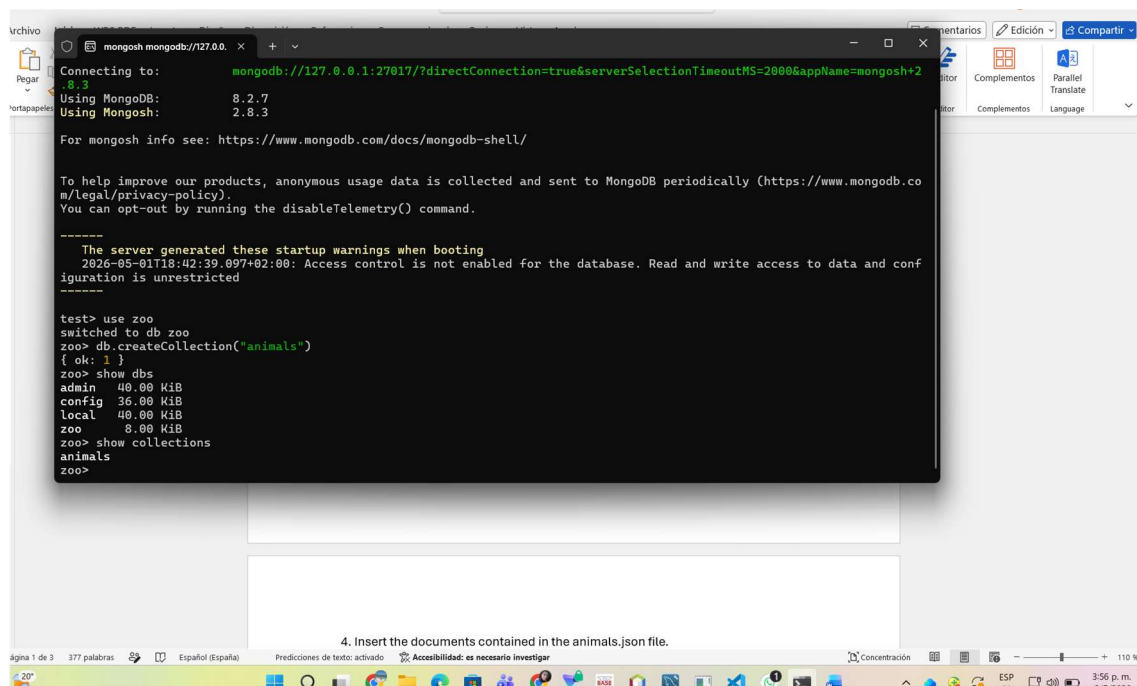
8. Write a MongoDB query to insert the next animal into the animals collection:

name: "Leo",

age: 7,

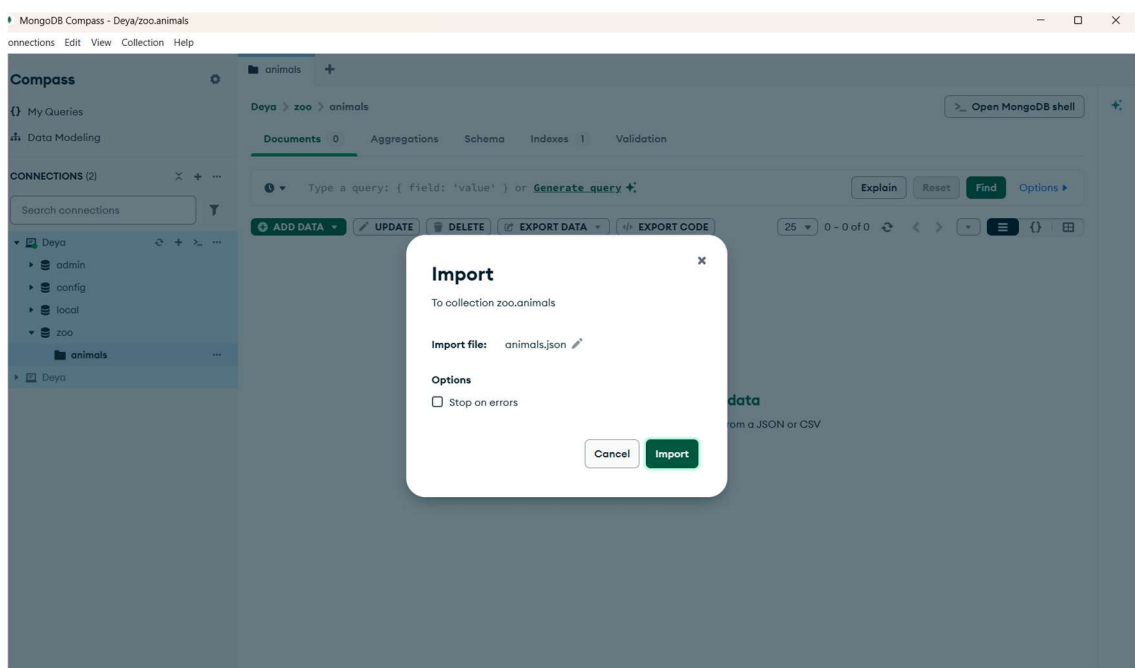
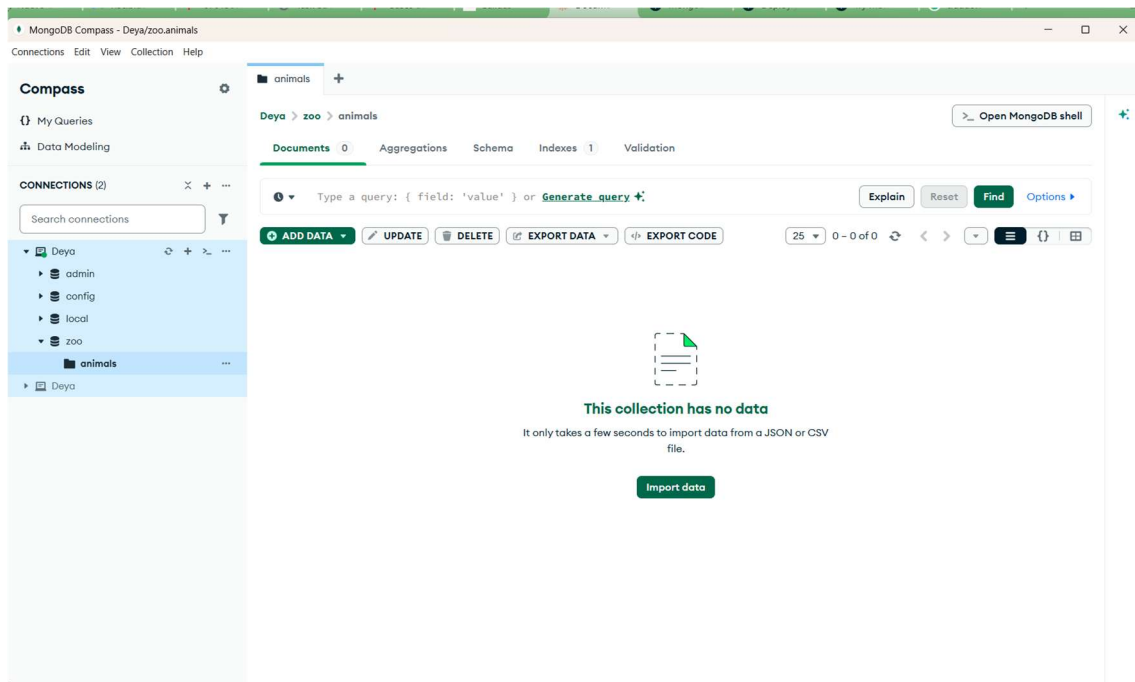
gender: "Male",

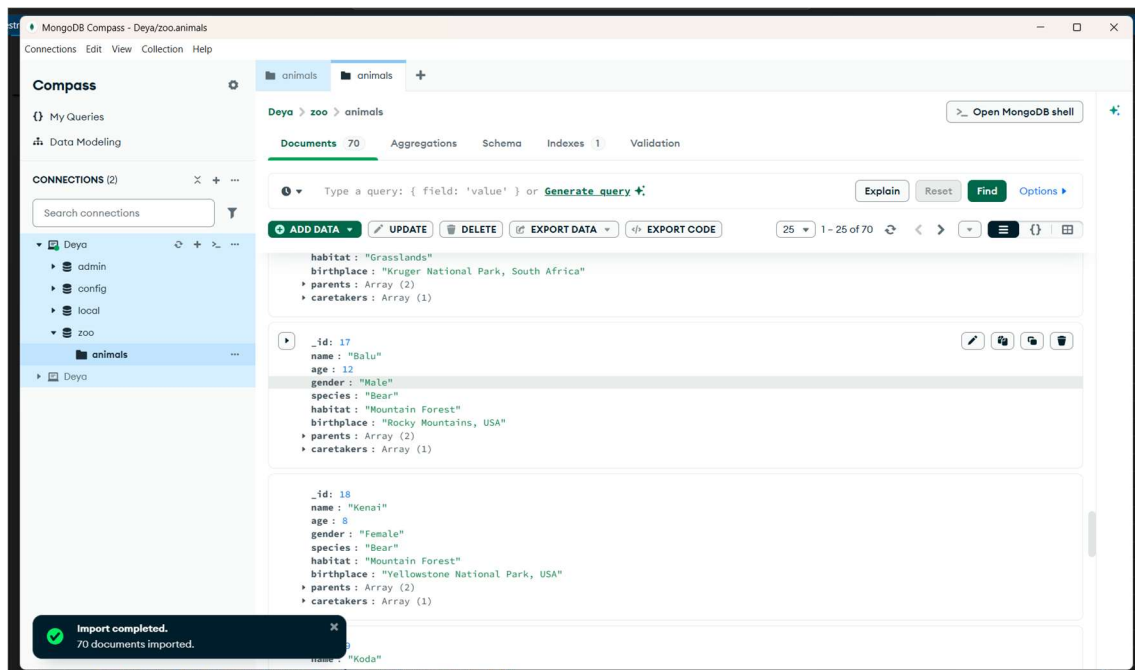
3. Confirm that the database and the collection have been successfully created.



4. Insert the documents contained in the animals.json file.

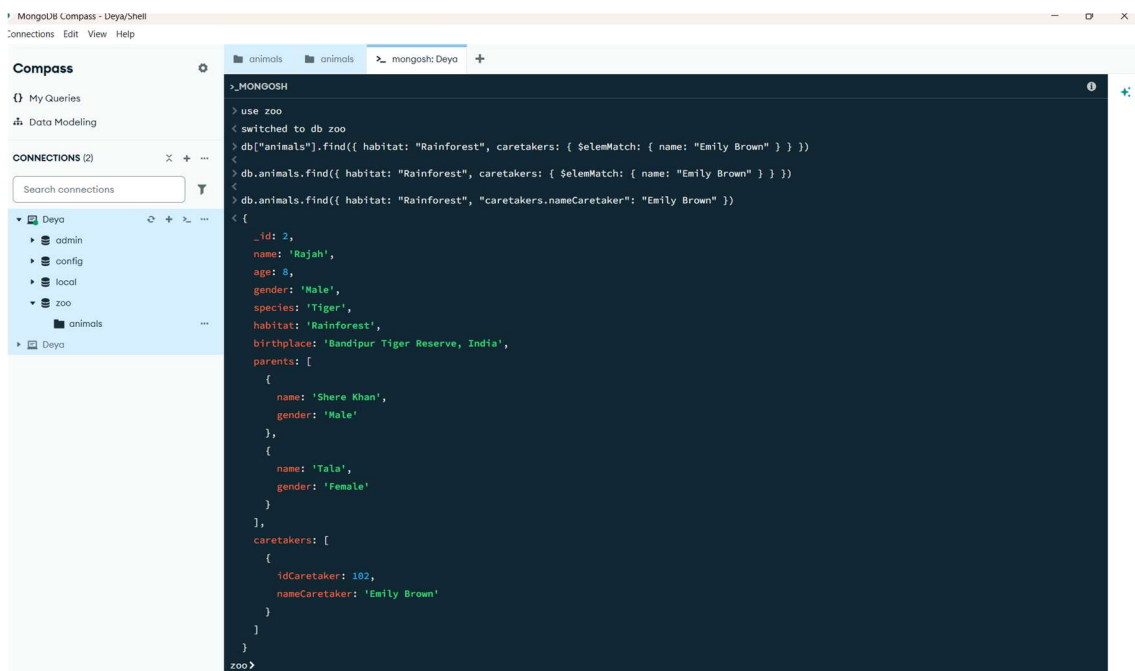
4. Insert the documents contained in the animals.json file.





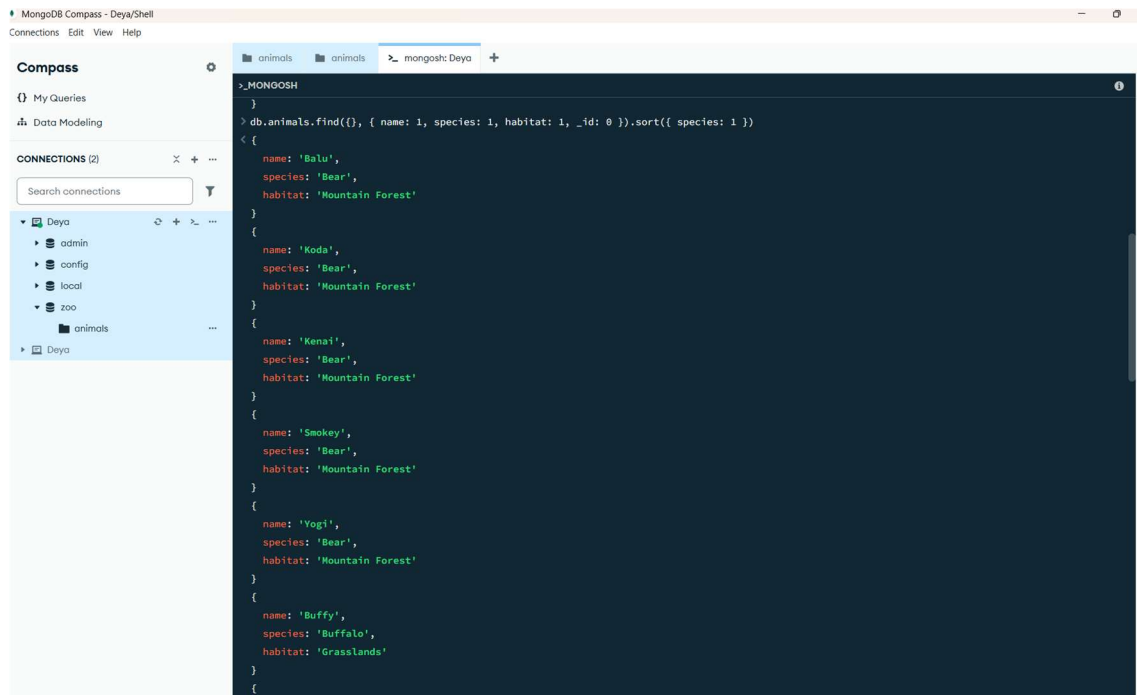
5- Write a MongoDB query to find animals that have Rainforest as their habitat and are attended by a caretaker named Emily Brown.

```
db.animals.find({ habitat: "Rainforest", "caretakers.nameCaretaker": "Emily Brown"
})
```



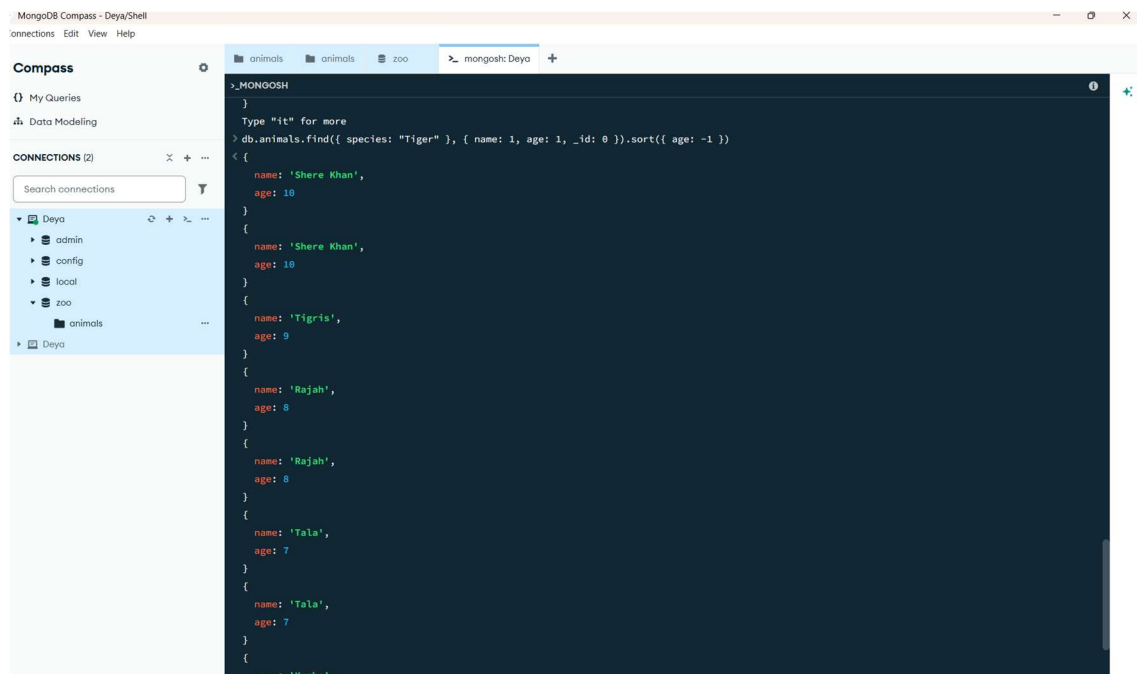
6- Write a MongoDB query to display only the name, species, and habitat of all animals sorted by their species in ascending order.

`db.animals.find({}, { name: 1, species: 1, habitat: 1, _id: 0 }).sort({ species: 1 })`



7. Write a MongoDB query to find animals with the species Tiger and display only their name and age, sorted by age in descending order.

`db.animals.find({ species: "Tiger" }, { name: 1, age: 1, _id: 0 }).sort({ age: -1 })`



8. Write a MongoDB query to insert the next animal into the animals collection:

name: "Leo",

age: 7,

```
gender: "Male",
species: "Lion",
habitat: "Savanna",
birthplace: "Masai Mara, Kenya",
parents: "Unknown", gender: "Male", "Unknown", gender: "Female"
caretakers: id: 501, name: "Jake Adams" }}
```

```
db.animals.insertOne({ name: "Leo", age: 7, gender: "Male", species: "Lion",
habitat: "Savanna", birthplace: "Masai Mara, Kenya", parents: [{ name: "Unknown",
gender: "Male" }, { name: "Unknown", gender: "Female" }], caretakers: [{
idCaretaker: 501, nameCaretaker: "Jake Adams" } ] })
```

```
db.animals.find({ name: "Leo" })
```

The screenshot shows a MongoDB CLI session. The first command is `db.animals.insertOne({ name: "Leo", age: 7, gender: "Male", species: "Lion", habitat: "Savanna", birthplace: "Masai Mara, Kenya", parents: [{ name: "Unknown", gender: "Male" }, { name: "Unknown", gender: "Female" }], caretakers: [{ idCaretaker: 501, nameCaretaker: "Jake Adams" } ] })`. The output shows the document was inserted with an `acknowledged` status of `true` and an `insertedId` of `ObjectId('69f62323ca9e40a7ce62a98f')`. The second command is `db.animals.find({ name: "Leo" })`, which returns a single document with the same fields as the inserted document.

9. Write a MongoDB query to update the age (to 8) of the animal you just inserted using the `$set` operator

```
db.animals.updateOne({name: "Leo"}, {$set: {age: 8}})
```

```

> db.animals.updateOne(
  { name: "Leo" },
  { $set: { age: 8 } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
zoo>

```

10. Write a MongoDB query to rename the habitat field to environment for all animals in the animals collection.

`db.animals.updateMany({}, {animals.updateMany({}, { $rename: {"habitat": "enviroment"} } )})`

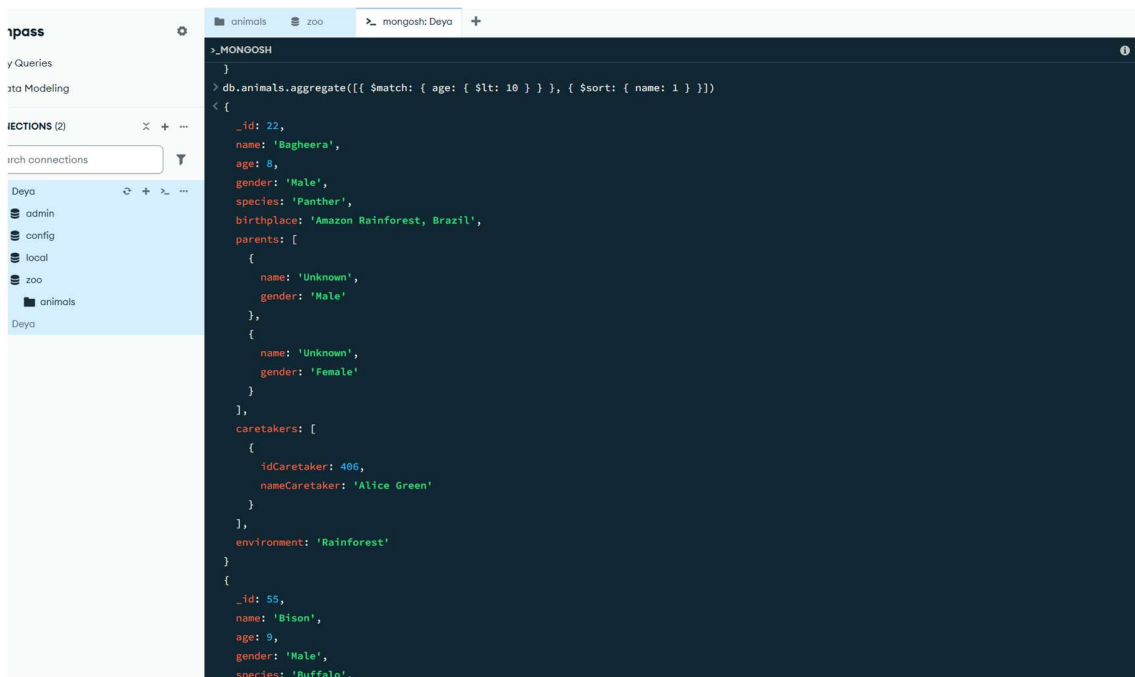
```

> db.animals.updateMany({}, { $rename: { "habitat": "environment" } })
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 71,
  modifiedCount: 71,
  upsertedCount: 0
}
> db.animals.findOne()
< {
  _id: 1,
  name: 'Shere Khan',
  age: 10,
  gender: 'Male',
  species: 'Tiger',
  birthplace: 'Sundarbans, India',
  parents: [
    {
      name: 'Unknown',
      gender: 'Male'
    },
    {
      name: 'Unknown',
      gender: 'Female'
    }
  ],
  caretakers: [
    {
      idCaretaker: 101,
      nameCaretaker: 'John Smith'
    }
  ]
}

```

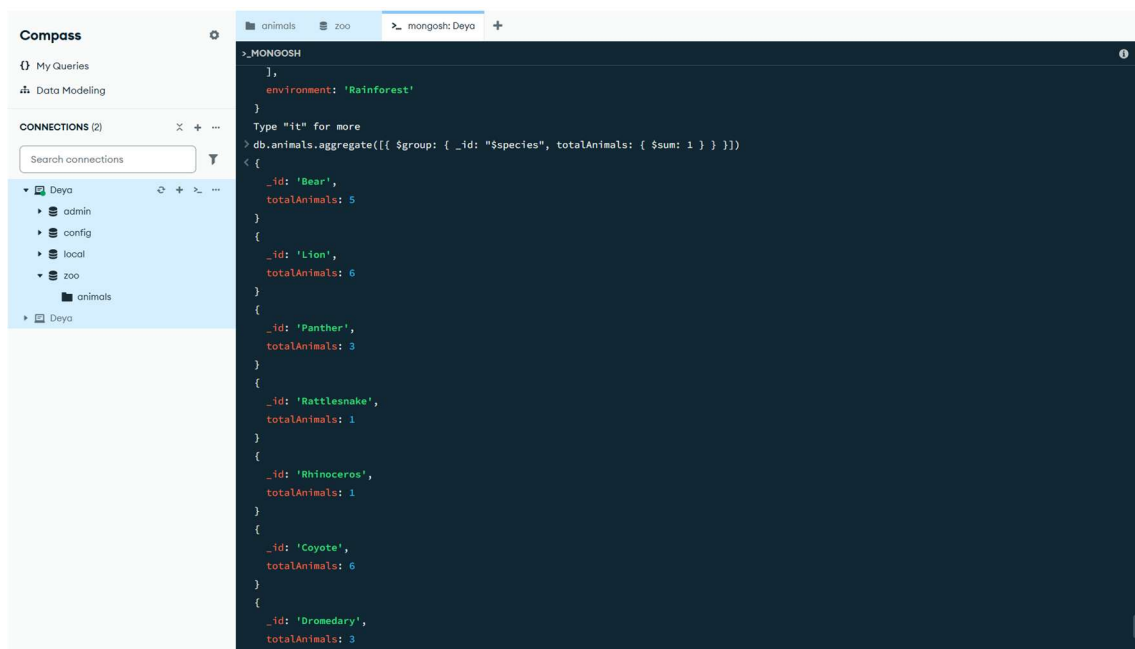
11- Write a MongoDB query using the aggregation framework to find animals aged less than 10 and sort them alphabetically by their name.

`db.animals.aggregate([{$match: { age: { $lt: 10 } }}, {$sort: { name: 1 } }])`



12. Write a MongoDB query using the aggregation framework to group animals by their species and count the number of animals in each species.

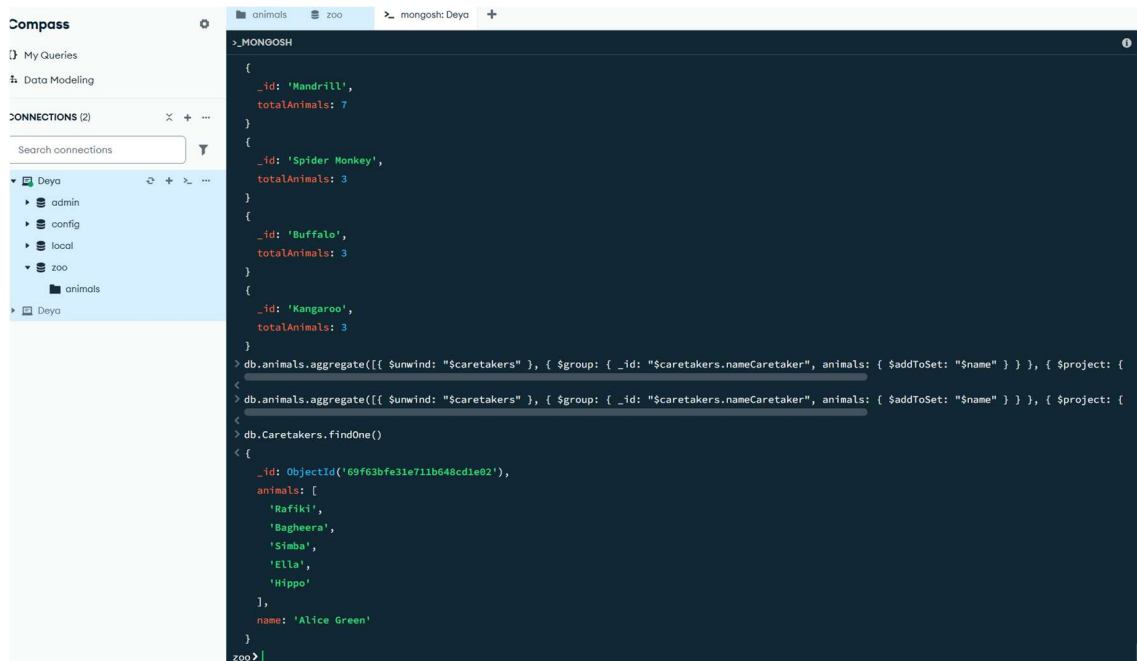
```
db.animals.aggregate([{$group: { _id: "$species", totalAnimals: { $sum: 1 } } }])
```



13. Write a MongoDB query to create a new collection called Caretakers by extracting all caretakers from the animals collection. Ensure that caretakers do not repeat and include an array of the animals they attend to.

```
db.animals.aggregate([{$unwind: "$caretakers"}, {$group: { _id: "$caretakers.nameCaretaker", animals: { $addToSet: "$name" } }}, {$project: { _id: 0, name: "$_id", animals: 1 }}, {$out: "Caretakers" }])
```

```
db.Caretakers.findOne()
```



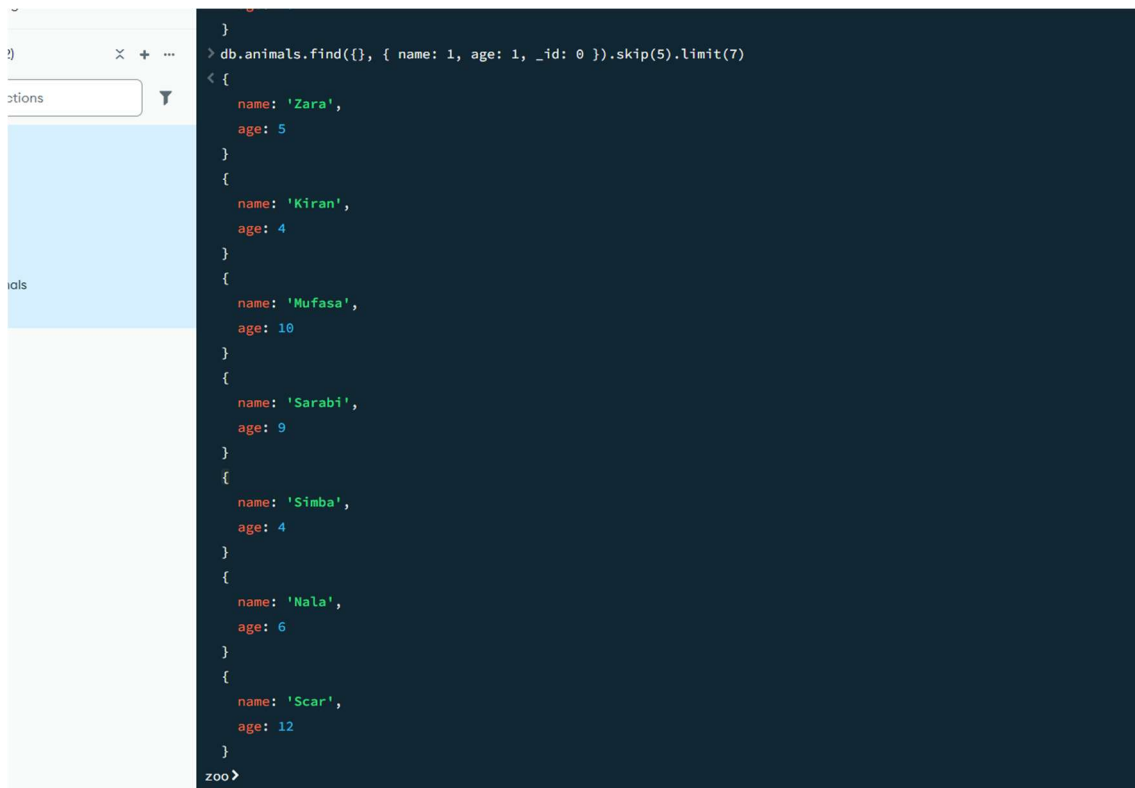
14. Write a MongoDB query to find the top 3 oldest animals.

```
db.animals.find({}, { name: 1, age: 1, _id: 0 }).sort({ age: -1 }).limit(3)
```



15. Write a MongoDB query to display animals between the 6th and 12th entries in the collection based on their insertion order.

```
db.animals.find({}, { name: 1, age: 1, _id: 0 }).skip(5).limit(7)
```

A screenshot of a MongoDB query result displayed in a terminal window. The query is `db.animals.find({}, { name: 1, age: 1, _id: 0 }).skip(5).limit(7)`. The result shows seven documents, each with a 'name' and an 'age' field. The documents are: {'name': 'Zara', 'age': 5}, {'name': 'Kiran', 'age': 4}, {'name': 'Mufasa', 'age': 10}, {'name': 'Sarabi', 'age': 9}, {'name': 'Simba', 'age': 4}, {'name': 'Nala', 'age': 6}, and {'name': 'Scar', 'age': 12}. The terminal window has a dark background with light-colored text. The prompt 'zoo>' is visible at the bottom left of the terminal area.

```
> db.animals.find({}, { name: 1, age: 1, _id: 0 }).skip(5).limit(7)
< {
  name: 'Zara',
  age: 5
}
{
  name: 'Kiran',
  age: 4
}
{
  name: 'Mufasa',
  age: 10
}
{
  name: 'Sarabi',
  age: 9
}
{
  name: 'Simba',
  age: 4
}
{
  name: 'Nala',
  age: 6
}
{
  name: 'Scar',
  age: 12
}
zoo>
```

16. Write a MongoDB query using the aggregation framework to create a new collection called Habitats. This collection should group animals by their environment (formerly habitat) and include the following fields.

- The environment name.
- The total number of animals in that environment.
- An array of unique species in that environment, sorted alphabetically.
- An array of unique caretakers who attend animals in that environment.

```
db.animals.aggregate([ { $unwind: "$caretakers" }, { $group: { _id: "$environment",
totalAnimals: { $sum: 1 }, species: { $addToSet: "$species" }, caretakers: {
$addToSet: "$caretakers.nameCaretaker" } } }, { $project: { _id: 0, environment:
"$_id", totalAnimals: 1, species: { $sortBy: { input: "$species", sortBy: 1 } },
caretakers: { $sortBy: { input: "$caretakers", sortBy: 1 } } } }, { $out: "Habitats" } ])
db.Habitats.find()
```

```
zoo> db.Habitats.find()
[
  {
    _id: ObjectId('679370346cee80a6809e83a7'),
    totalAnimals: 11,
    environment: 'Grasslands',
    species: [ 'Buffalo', 'Elephant', 'Kangaroo', 'Rhinoceros' ],
    caretakers: [
      'Alice Brown',
      'Alice Green',
      'Alice White',
      'Emily Brown',
      'Emily Green',
      'Jake Adams',
      'John Smith',
      'Mark Taylor',
      'Nathan Hall'
    ]
  },
  {
    _id: ObjectId('679370346cee80a6809e83a8'),
    totalAnimals: 10,
    environment: 'Savanna',
    species: [ 'Giraffe', 'Lion' ],
    caretakers: [
      'Alice Green',
      'Emily Brown',
      'Jake Adams',
      'John Smith',
      'Mark Taylor',
      'Michael Davis',
      'Sophia Green'
    ]
  },
  {
    _id: ObjectId('679370346cee80a6809e83a9'),
    totalAnimals: 3,
    environment: 'River',
    species: [ 'Hippopotamus' ],
    caretakers: [ 'Alice Green', 'Mark Brown', 'Sophia Taylor' ]
  },
]
```

mpass

My Queries

Data Modeling

INJECTIONS (2)

search connections

Deya

admin

config

local

zoo

animals

Deya

animals

zoo

mongosh: Deya

>_MONGOSH

```
_id: ObjectId('69f6421431e711b648cd1e24'),
totalAnimals: 2,
environment: 'Mountains',
species: [
  'Eagle'
],
caretakers: [
  'Emily Brown',
  'John White'
]
}
{
  _id: ObjectId('69f6421431e711b648cd1e25'),
totalAnimals: 10,
environment: 'Desert',
species: [
  'Coyote',
  'Dromedary',
  'Rattlesnake'
],
caretakers: [
  'Emily Brown',
  'Emily White',
  'Jake Adams',
  'John White',
  'Mark Taylor',
  'Nathan Green',
  'Nathan Hall',
  'Sophia Green',
  'Sophia Taylor'
]
}
zoo>
```